

第 20 届湖南省大学生计算机程序设计竞赛

-----应用开发类竞赛（2024）

系
统
设
计
说
明
书

作品名称：面向 AI 换脸图像伪造的双重水印防御系统

作品类别：信息安全类

作 者：晏碧涛、吕志、夏海涛

指导老师：陈纪友、张剑

单 位：衡阳师范学院

二零二四年八月三十日

目 录

1 功能需求.....	1
1.1 研究背景与意义.....	1
1.2 研究现状.....	2
2 概要设计.....	3
2.1 数字水印的定义.....	3
2.2 数字水印的特点.....	3
2.3 数字水印的分类.....	4
3 算法详细设计	5
3.1 数字水印的嵌入算法设计	5
3.2 数字水印的提取算法设计	9
3.3 图像攻击的算法设计	12
3.4 人脸分割算法设计.....	16
4 界面设计.....	17
4.1 GUI 界面的设计.....	17
4.2 系统的测试与评估.....	21
4.3 系统上传功能测试.....	21
4.4 系统嵌入与提取水印功能测试.....	22
4.5 系统攻击功能测试.....	24
4.6 测试结果与评估.....	25
4.7 小结.....	26
5 用户操作简介	27
5.1 设置密码功能.....	27
5.2 图像加载功能.....	27
5.3 水印自定义功能.....	29
5.4 嵌入水印功能.....	29
5.5 验证水印功能.....	31
5.6 保存图片.....	31
5.7 图像攻击测试功能.....	32
5.8 AI 换脸工具.....	33



1 功能需求

1.1 研究背景与意义

2023 年 2 月 27 日至 28 日,第七届中国网络版权保护与发展大会于四川成都顺利举行,会议上表明了以 ChatGPT 为代表的生成人工智能大模型的快速发展给版权保护带来了前所未有的挑战。根据数据的粗略统计,截至 2023 年底,中国网络数字作品市场规模高达 404 亿元。随着互联网产品技术和服务模式的迅速演进,新型的数字作品盗版侵权模式层出不穷,甚至逐渐构建起产业链条,这种盗版侵权行为严重扰乱了市场秩序,对行业的健康发展造成严重影响。完善数据版权保护的法律法规以及推动数字版权保护的科技创新与应用变得愈发重要。

随着人民生活水平质量的提高,人们对于精神上的需求变得多了起来,因而摄影、绘画、ACG 等行业变得流行起来。为了防止作品被盗用,创作者们大部分都会为自己的作品添加一个标识,用来表明这副作品是由该作者创作的,这个标识即现在人尽皆知的水印。传统的普通的水印很容易就能够在作品中被找到,且经过人为的裁剪、修图或是遮挡就能轻易的去除作品的水印,从而达到盗取的目的。而数字水印正好可以一定程度上解决这些问题。数字水印技术起初主要应用于音频领域,用于保护数字音频内容的版权。正如前面所提到的,近几年来数字图像的发展非常迅猛且应用十分广泛,为减少盗版侵权事件的发生,数字水印技术逐渐被打磨开发,开始用于图像领域,并成为了信息安全领域的热门研究方向。

对于数字水印技术的意义主要还是用于版权保护,数字水印可以嵌入到数字媒体文件中,以证明文件的所有权和来源。这对于保护图像、音乐、视频等数字创作作品的版权具有重要意义,帮助制作者和创意产业维护其合法权益;数字水印可以用于验证数字媒体文件的完整性和真实性。通过检查水印,人们可以确定文件是否被篡改或操纵,从而确保数字内容的可信度。在商业领域,数字水印可以嵌入到产品图像、设计或标识中,以防止伪造和仿冒。使用较为广泛的是在物品上嵌入唯一标识,以帮助追踪和识别物品的真伪或来源。数字水印技术有助于提高数字内容的安全性,防止未经授权的复制和分发。这对于电子书、音乐、电影等数字化娱乐内容的产业具有重要意义,有助于减少盗版问题。在法律和法庭环境中,数字水印还可以用作证据,帮助追踪和确定媒体文件的来源。这在打击网络犯罪和数字侵权方面发挥着关键作用。

总体而言,数字水印技术的应用为数字化时代的信息安全、知识产权保的发展提供了有效手段,现阶段也已经有较为成熟的数字水印技术,在图像中嵌入数字水印的作品与原始图像相对比,甚至不能感觉出图像中嵌入的数字水印,而只能特殊的通过解密算法才能够提取出相应的水印信息。数字水印技术极大的促进了数字媒体的合法使用和可持续发展。



1.2 研究现状

现有的关于水印技术的实现大多数都是通过直接添加感知性较高的水印来实现的,这种水印技术有一个很明显的缺陷,即不能经受任何攻击,图像的安全性在攻击者看来几乎为零,通过简单的操作便能够得到去除水印的图像。而如今对于水印的研究已经不再局限于传统普通水印,而是在不断深入探究具有高安全性和低感知性的数字水印。

对于图像数字水印技术的历史,相对其他技术来说算是比较晚的一种。最早对于数字水印的研究可以追溯到 1993 年 Tirkel 等人提出的 LSB 图像隐写技术,通过把图像信息转化为二进制形式,然后进行逐像素的处理,通过遍历载体图像的每个像素并对其灰度值的最不显著位(即最低有效位)上进行修改二进制的操作,从而达到嵌入水印信息的目的。这种方法优点在于非常容易实现,但缺点也很明显,是感知性最高且安全性最低的一种方法。

国内对于数字水印技术这个领域展开研究工作起步较晚。1999 年,全国信息隐藏讨论会成功举办,何德全院士和来自各院校的专家学者对信息隐藏技术首次展开讨论。往后几年国内对于数字水印技术的研究不温不火,只有 2001 年刘瑞祯等人研究出的将水印叠加到原始图像的 SVD 中,从而实现鲁棒性数字水印。到 2007 年,各地专家学者开始了对于数字水印研究的热潮。刘俊景团队^[6]提出基于离散余弦变化与奇异值分解的数字图像水印算法;陈希等人^[7]提出的基于 SVD 分解的小波域数字水印算法;还有基于傅里叶变换的数字水印 (FFT)、基于离散小波域的数字水印 (DWT) 等。这些技术都具有不错的鲁棒性、安全性和隐蔽性。总体来说国内对于数字水印技术的研究愈发重视。

而现如今,国内外对于数字水印技术的研究研究工作开始和深度学习开始相关联。一些学者开始使用卷积神经网络 (CNN)、生成对抗网络 (GAN)、端到端学习、迁移学习等深度学习方法来实现数字水印技术。这些方法可以通过学习大量图像数据来提高水印的鲁棒性和隐蔽性,且还具有十足的安全性。未来数字水印技术的研究方向将主要围绕优化数字水印算法,提高数字水印嵌入和提取技术的效率和准确性,以及提高数字水印的安全性和防篡改技术来研究。



2 概要设计

2.1 数字水印的定义

数字水印是一种嵌入在数字媒体中的隐藏信息，用于验证媒体的真实性、完整性和所有权。它类似于传统的普通水印，但数字水印是通过在作品数据中嵌入特定的信息来实现的，且不同于传统水印的是，数字水印是不可见的，事先并不了解水印的内容。数字水印通常用于图像、音频、视频等数字媒体文件。本文所讨论的图像数字水印自然就是嵌入于图像中的一种水印。其最主要的用途主要在版权保护领域。

2.2 数字水印的特点

(1) 隐蔽性：在数字水印中的首要特性就是隐蔽性，隐蔽即字面意思，即人们能感知到图像中是否嵌入了信息的能力。当数字水印嵌入到图像中时，由于嵌入算法会对图像的一些参数值做一些修改，对用户而言，多少是能够感知到图像经过了某种变换。不良的嵌入算法设计甚至有时会破坏人们对于图像的观感体验。而高的隐蔽性能让人们仅凭借眼球几乎无法捕获到原图像与嵌入水印图像的区别。

(2) 鲁棒性：鲁棒性用简单的话术来表示，即为经过攻击后能够从水印图像中提取出水印的能力。水印的攻击手法市面上有非常多，常见的有如压缩、裁剪、旋转等在图像上进行的操作。鲁棒性是确保水印稳健性高低的衡量标准。

(3) 安全性：数字水印应该需要具有一定的安全性能。数字水印的算法有很多类型，其中比较简单的水印算法只需要提供原图像与水印图像就能够破解出嵌入的水印，从而对水印进行移除甚至可以替换为未授权的水印。加密和认证技术通常用于增强数字水印的安全性，以防止对水印的恶意攻击。

(4) 容量：图像中最多能够嵌入的信息量。数字水印的容量一般来说都相对较小，但优秀的数字水印算法是一定需要提供足够的容量来支持所需的身份验证或版权保护功能。

(5) 唯一性：数字水印应当是唯一的，一个数字水印图像只能嵌入一个水印标识，以区分不同的图像。

(6) 实时性：一些应用场景可能需要实时性，即对图像进行水印嵌入和提取的速度要足够快，以适应实时需求，比如实时版权检测。

(7) 适应性：数字水印技术应该需要适应不同类型的图像，包括分辨率、色彩空间和格式等方面的变化。



2.3 数字水印的分类

图像数字水印其实可以细分为很多类型,根据嵌入的方式、水印的实现算法、水印特性以及应用领域的不同可以进行多种分类。这里主要以不同水印的实现算法来对图像数字水印进行分类,并介绍其中具有代表意义的算法水印,具体如下:

(1) 频域水印: 频域,即信号在频率上的特性,提供了信号在不同频率上的分解和表达。在频域分析中,信号可以通过一些变换技术转换为频谱,其中包含了信号中各种不同频率的分量。最常见的变换技术为傅里叶变换。得到频谱后会对频谱进行分析,在频域中选择一些特定的频率分量,通常会选择低频区域,以确保水印的稳定性和不易被察觉。而最后这些所选择的频率分量中就会被嵌入水印。这种算法通常会在频域图像中加入一些微小的变化,而这些变化对于用户肉眼来说几乎是不可见的,而在后续检测中能够被提取。至于提取过程即为嵌入过程的逆过程^[10]。

(2) 空域水印: 空域在图像处理中指的是图像的原始表示,即像素级别的表示。空域处理是直接在图像的空间域上进行操作的。这些操作主要涉及到像素值的修改、滤波和其他基本图像处理技术。而空域水印技术通常会将图像分成小块,在特定块中嵌入水印信息。具体可以通过修改像素值或微调颜色分量来实现水印的嵌入。但要注意的是嵌入时要确保对图像的影响最小,以保证图像质量,从而不至于对观感有太大的影响。提取过程同样为嵌入过程的逆过程。

(3) 变换域水印: 图像处理中的变换域是指将图像从其原始的空间域转换到另一种数学表示的域。这种变换有助于在不同视角下理解和处理图像,以及实现一些特定的图像处理目标。最常见的变换域有傅里叶变换、小波变换。前者是将图像从空间域转换到频率域,显示图像中不同频率的分量,常用于频率滤波、图像压缩和特征提取等任务;而后者是将图像转换到小波域,以显示图像中不同尺度和方向的信息,常用于多尺度分析、去噪和图像压缩等应用。而数字水印的嵌入就是在变换域的类型后修改对应的系数从而达到嵌入的目的。嵌入和提取过程与前面介绍的两种水印算法类似^[11]。

(4) 深度学习水印: 深度学习数字水印是利用深度学习技术嵌入和提取数字水印的一种方法。这种方法通常使用卷积神经网络(CNN)或其他深度学习架构,以学习图像中嵌入数字水印的特征,这些方法具有较强的鲁棒性和安全性。在深度学习数字水印的过程中,模型被训练用于将数字水印嵌入到图像中。通常,这涉及将水印信息融入图像的特定层或通道中,以确保水印能够在深层表示中得到保留。深度学习方法可以通过大量数据的学习来提高对各种攻击和变换的鲁棒性,包括压缩、旋转、缩放、噪声等。

3 算法详细设计

在详细介绍之前需要说明一下的是本系统采用了 RiVAGAN 数字水印算法设计，并对其进行了结合与优化。另外该系统对嵌入以及提取算法还提供了盲水印策略。盲水印策略，即数字水印的载体作品只是作为载波信号而存在，不会向水印检测器提供任何的有效信息，即提取过程不会使用到原图，只需要使用嵌入了数字水印的作品。

3.1 数字水印的嵌入算法设计

首先介绍的即为数字水印的嵌入，嵌入过程如图 3-1 所示。

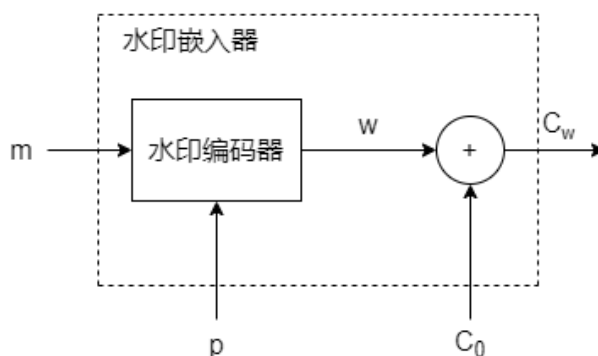


图 3-1 数字水印嵌入的简单流程

其中 m 为原始水印信息； p 为水印密钥； w 为水印编码，即由 m 经过水印编码器生成的编码流； C_0 为载体作品，即原图； C_w 为生成的含数字水印信息的水印作品。

水印信息通过水印编码器中包含的水印嵌入算法将水印信息转化为水印编码流。其中在转化的过程中可以选择性的加入水印密钥，从而使得生成的水印编码流更具安全性。当然，不添加水印密钥同样也是可行的。生成水印编码流后，就可以将其通过嵌入算法嵌入到载体作品中。最终会生成出相应的含数字水印的水印作品。

在这里对于算法中的数值运算和图像处理分别使用了 Python 中的 Numpy 库和 OpenCV 库来处理。首先会对载体作品进行初始化，计算图像中可用于嵌入水印的块的数量，通过读取指定的载体作品，并将其转换为 YUV 色彩空间^[12]，从而对像素进行调整，将像素数值加白偶数化，即调整图像的像素值，使其都变为偶数。之后便能够进行水印的循环嵌入了。对通道进行二维的 Haar 小波变换处理^[13]，从而获得低频和高频部分，然后对低频部分进行再转化，将所得的系数重新排列成表示为块的四维数组。之后就是水印的嵌入操作，初始化水印信息后，然后根据水印数据的比特位进行修改。对混洗过后的块进行奇异值分解(SVD)以进行水印嵌入。将修改后的块进行逆变换就能够获得到嵌入水印的图像。具体的算法设计如下：



(1) 图像预处理

首先，当用户按下嵌入按钮的时候，我们需要获取到对应图像以及对应输入的变量。会有一个名为 `read_img` 的方法，用于读取载体作品文件，之后会直接对作品进行处理，具体代码如下所示。

```
def read_img(self, filename):
    img = cv2.imread(filename)
    if img is None:
        raise IOError("图像'{filename}'未读取到".format(filename=filename))
    self.img = img.astype(np.float32)
    self.img_shape = self.img.shape[:2]
    self.img_YUV = cv2.cvtColor(self.img,
                                cv2.COLOR_BGR2YUV),
                                0, self.img.shape[0] % 2, 0, self.img.shape[1] % 2,
                                cv2.BORDER_CONSTANT, value=(0, 0, 0))
    self.ca_shape = [(i + 1) // 2 for i in self.img_shape]
    self.ca_block_shape = (self.ca_shape[0] // self.block_shape[0],
                           self.ca_shape[1] // self.block_shape[1],
                           self.block_shape[0], self.block_shape[1])
    strides = 4 * np.array([self.ca_shape[1] * self.block_shape[0],
                           self.block_shape[1], self.ca_shape[1], 1])
    for channel in range(3):
        self.ca[channel], self.hvd[channel] = dwt2(self.img_YUV[:, :, channel], 'haar')
        self.ca_block[channel] = np.lib.stride_tricks.as_strided(self.ca[channel].astype(np.float32),
                                                                    self.ca_block_shape, strides)
```

这段代码主要是对于读入的图像进行处理，使用 `img` 存放了使用 `astype` 转换而来的 `float32` 类型的数据，这样有利于后续算法中对图像的处理；使用 `img_shape` 存放了图像的尺寸数据，使用切片操作 `[:2]` 来获取前两个元素，即高度和宽度；对于读入的图像因为要利用其 `YUV` 特性，所以必须使其 `YUV` 化，因为图像默认的颜色空间为 `RGB` 模式，所以先要使用 `cvtColor` 方法来将颜色空间转换为 `YUV` 模式，另外假若图像的宽和高不为偶数，则会使用 `copyMakeBorder` 为图像边缘添加白边，使得图像的宽度和高度都变为偶数，经过转换后的图像数据会使用 `img_YUV` 来存储；`ca_shape` 用于存储 `YUV` 通道的尺寸，因为在后续的处理中会对 `YUV` 通道进行采样，所以尺寸会减半；使用 `ca_block_shape` 来存储 `YUV` 通道的分块形状；使用 `strides` 来存储计算出的思维分块的步幅；最后对图像的每个颜色通道(Y、U、V)执行以下操作：`dwt2`，对当前颜色通道进行二维离散小波变换(DWT)；`as_strided`，根据给定的步幅创建一个多维数组视图，用于表示图像的小波系数分块。

经过对载体作品各个参数的读取、计算和转换后，整个图像的初始化就完成了。



(2) 模式判断

对于水印的读取要分为文字水印以及图像水印，这里是采用模式的方式来分出的情况，具体代码如下所示。

```
def read_img_wm(self, filename):
    wm = cv2.imread(filename)
    if wm is None:
        raise IOError("图像'{filename}'未读取到".format(filename=filename))
    self.wm = wm[:, :, 0]
    self.wm_bit = self.wm.flatten() > 128

def read_wm(self, wm_content, mode='img'):
    if mode == 'img':
        self.read_img_wm(filename=wm_content)
    elif mode == 'str':
        byte = bin(int(wm_content.encode('utf-8').hex(), base=16))[2:]
        self.wm_bit = (np.array(list(byte)) == '1')
    else:
        self.wm_bit = np.array(wm_content)
    self.wm_size = self.wm_bit.size
```

对于水印的处理则比较简单了，假若模式为 `str` 即为文字水印，直接将输入的文字水印内容转换为二进制形式，并在最后记录水印信息的大小；假若模式为 `img` 即为图像水印，调用 `read_img_wm` 方法，读取图像的通道，即灰度通道，因为加密信息只用二进制数据，所以需要抛弃灰度级别，当像素值大于 128 时则视为 1，小于等于 128 时则视为 0。

(3) 水印嵌入

既然载体和水印都已经初始化完成，紧接着就是对于水印的嵌入了。数字水印的嵌入过程是整个过程中最重要的部分之一，前面所做的诸如初始化载体作品与初始化图像水印或文字水印的操作都是为了这一步水印的嵌入操作。而 GUI 所调用的嵌入水印的操作也正是这一步。因此这一环的重要性可想而知，其具体代码如下：

```
def init_block_index(self):
    self.block_num = self.ca_block_shape[0] * self.ca_block_shape[1]
    assert self.wm_size < self.block_num, IndexError(
        '最多可嵌入{}kb 信息，多于水印的{}kb 信息，溢出'
        .format(self.block_num / 1000, self.wm_size / 1000))
```

这一步是为了初始化可嵌入的块数、每个分块的大小以及分块的索引。这些参数都将用于后续的水印嵌入过程中。其中要注意的是 `part_shape` 是取整后的 `ca` 二维大小，用于嵌入时忽略右边和下面对不齐的细条部分。



```
def random_strategy(seed, size, block_shape):  
    return np.random.RandomState(seed).random(size=(size,  
                                                    block_shape)).argsort(axis=1)
```

`random_strategy` 是一个简单的随机策略方法, 用于生成一个随机的索引重排列, 最终会返回一个形状为 $(size, block_shape)$ 的二维数组, 其中每一行都是对应分块的随机重排序索引。这个索引数组将在水印嵌入过程中使用, 用于对分块的小波系数进行重新排序。

```
def block_add_wm(self, arg):  
    block, shuffler, i = arg  
    wm_1 = self.wm_bit[i % self.wm_size]  
    block_dct = cv2.dct(block)  
    block_dct_shuffled = block_dct.flatten()[shuffler].reshape(self.block_shape)  
    U, s, V = np.linalg.svd(block_dct_shuffled)  
    s[0] = (s[0] // self.d1 + 1 / 4 + 1 / 2 * wm_1) * self.d1  
    if self.d2:  
        s[1] = (s[1] // self.d2 + 1 / 4 + 1 / 2 * wm_1) * self.d2  
    block_dct_flatten = np.dot(U, np.dot(np.diag(s), V)).flatten()  
    block_dct_flatten[shuffler] = block_dct_flatten.copy()  
    return cv2.idct(block_dct_flatten.reshape(self.block_shape))
```

`block_add_wm` 这个方法用于将水印信息嵌入到给定的图像块中。方法中会对输入的图像块进行 DCT 变换, 得到其 DCT 变换系数 `block_dct`, 然后会根据之前获得到的密码对 DCT 变换系数进行加密操作, 整个过程中涉及到重排序、奇异值分解(SVD), 并会根据水印比特位调整奇异值, 最后恢复重排的顺序。加密后的 DCT 变换系数, 通过逆 DCT 变换能够得到水印信息嵌入后的图像块, 最后将返回嵌入水印信息后的图像块。整个水印信息的嵌入实际上就是在这里完成的。

```
def embed(self, filename):  
    self.idx_shuffle = random_strategy1(self.password_img,  
                                       self.block_num, self.block_shape[0] * self.block_shape[1])  
    for channel in range(3):  
        tmp = self.pool.map(self.block_add_wm,  
                            [(self.ca_block[channel][self.block_index[i]],  
                             self.idx_shuffle[i], i) for i in range(self.block_num)])  
        for i in range(self.block_num):  
            self.ca_block[channel][self.block_index[i]] = tmp[i]  
            self.ca_part[channel] = np.concatenate(np.concatenate(  
                                                    self.ca_block[channel], 1), 1)  
            embed_ca[channel][:self.part_shape[0],  
                             :self.part_shape[1]] = self.ca_part[channel]  
            embed_YUV[channel] = idwt2((embed_ca[channel], self.hvd[channel]), "haar")
```

嵌入水印过程的主体是 `embed` 方法。上述代码中使用了先前介绍的 `init_block_index` 方



法用于初始化块索引，然后深度拷贝原始的小波系数并创建一个空的 `embed_YUV` 列表用于存储嵌入水印后的 YUV 颜色通道；之后使用随机策略来生成索引重新排序列。

接下来就会对图像的每个颜色通道分块并调用前面介绍到的 `block_add_wm` 方法，将水印信息嵌入到相应的小波系数分块中后会将嵌入了水印信息的小波系数分块恢复为二维比特形式，并最后逆变换为图像空间的 YUV 颜色通道。

后面的操作即是用来将颜色通道合并然后通过一系列操作转化为一张图像。至此，水印的嵌入过程结束。

3.2 数字水印的提取算法设计

数字水印的提取过程，在嵌入过程的基础上，过程如图 3-2 所示。

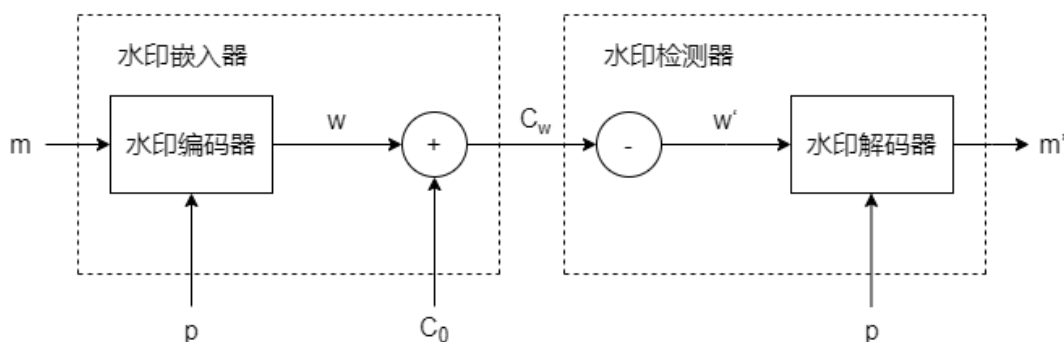


图 3-2 数字水印提取的简单流程

其中 w' 为嵌入数字水印图像的水印编码； m' 为提取出的水印信息。

含数字水印的图像经过读取能够获取到相应的水印编码流，通过水印解码器中包含的算法对水印编码流进行解码，若有水印密钥同样能够进行联合解码。最终能够获得相应的水印信息。

对于水印的提取，其实为嵌入的逆过程。首先需要读取指定的图像文件，对其进行 YUV 转换和分块处理，从而得到相应的块信息。接着会去提取水印流，对每个通道的每个块都进行 DCT 和 SVD，从中提取出水印比特流。将每个通道和块的水印流经过整理后转化为数组后，最后对其在通道和块方向上进行平均就能够获得相应的水印信息。

在讨论数字水印时，通常会把嵌入水印和提取水印一同考虑。而在前面已经介绍过嵌入水印算法的设计了，这里将详细介绍数字水印的提取过程。

实际上，数字水印的提取过程在清楚数字水印的嵌入过程后就会比较容易理解了。数字水印的提取过程就相当于数字水印嵌入过程的逆过程。对于 GUI 中的提取水印按钮，当按下提取水印按钮时，所调用的函数即如下所示。

```
def extract(self, filename, wm_shape, out_wm_name=None, mode='img'):
```



```
self.wm_size = np.array(wm_shape).prod()
if mode in ('str', 'bit'):
    wm_avg = self.bwm_core.extract_with_kmeans(filename=filename,
wm_shape=wm_shape)
else:
    wm_avg = self.bwm_core.extract(filename=filename, wm_shape=wm_shape)
wm = self.extract_decrypt(wm_avg=wm_avg)
if mode == 'img':
    cv2.imwrite(out_wm_name, 255 * wm.reshape(wm_shape[0], wm_shape[1]))
elif mode == 'str':
    byte = ".join((np.round(wm)).astype(np.int64).astype(np.str_))
    wm = bytes.fromhex(hex(int(byte, base=2))[2:]).decode('utf-8')
return wm
```

提取过程同样，是需要先获取一些参数变量的。最首要应当知道的为提取的是文字水印还是图像水印。而这个参数 `mode` 将通过用户点击不同的按钮来变为不同的值，其值为 `'img'` 或 `'str'`，即所对应图像水印或文字水印。

在具体展开文字水印或图像水印的提取前，还需要对用户指定的水印图像进行预处理，由于资源初始化的代码较长，将在以下分块来逐一介绍。

```
def extract_raw(self, filename):
    self.read_img(filename)
    self.init_block_index()
    wm_block_bit = np.zeros(shape=(3, self.block_num))
    self.idx_shuffle = random_strategy1(seed=self.password_img,
size=self.block_num,
block_shape=self.block_shape[0] * self.block_shape[1], )
    for channel in range(3):
        wm_block_bit[channel, :] = self.pool.map(self.block_get_wm,
[(self.ca_block[channel][self.block_index[i]],
self.idx_shuffle[i]) for i in range(self.block_num)])
    return wm_block_bit
```

`extract_raw` 方法实际上与水印嵌入算法使用了共同的方法，因此过程也大体类似。在读取图像数据以及初始化分块索引后，通过 `wm_block_bit` 变量来存储每个通道提取的水印信息。接着同样使用了随机策略对分块的数据进行乱序。最后遍历 RGB 三个通道，对每个分块数据调用 `block_get_wm` 方法进行水印信息提取，最后将提取的结果返回。

```
def block_get_wm(self, args):
    block, shuffler = args
```



```
block_dct_shuffled = cv2.dct(block).flatten()[shuffler].reshape(self.block_shape)
U, s, V = np.linalg.svd(block_dct_shuffled)
wm = (s[0] % self.d1 > self.d1 / 2) * 1
```

对于 `block_get_wm` 方法，则类似于嵌入水印算法中的 `block_add_wm`。首先对分块数据进行离散余弦变换(DCT)，然后将结果展平，并按照给定的乱序索引进行重新排列。然后将分块数据进行奇异值分解(SVD)，分解为 Y、U、V 三个矩阵。之后便提取第一个奇异值并进行一定的计算，最终可以得到水印信息，然后继续进行判断是否存在第二个奇异值，若存在即可得到临时水印信息，将第一个水印信息与临时水印信息采用取加权平均值的方法最终即可获得水印信息 `wm`。

```
def extract_avg(self, wm_block_bit):
    wm_avg = np.zeros(shape=self.wm_size)
    for i in range(self.wm_size):
        wm_avg[i] = wm_block_bit[:, i::self.wm_size].mean()
    return wm_avg
```

对于 `sxtract_avg` 方法，是用于对提取的原始水印信息进行平均处理的。该方法将会遍历水印信息数组的每个元素，进行平均处理。

```
def extract(self, filename, wm_shape):
    self.wm_size = np.array(wm_shape).prod()
    wm_block_bit = self.extract_raw(filename=filename)
    wm_avg = self.extract_avg(wm_block_bit)
    return wm_avg
```

对于 `extract` 方法，与上面介绍的同名方法有所不同，它调用了之前介绍的 `extract_raw` 方法以及 `extract_avg` 方法最终将返回处理后的水印信息数组。

介绍完以上内容，对于图像水印的水印信息就能够提取出来了，此时还是比特流的形式存放在数组中。而 `extract_decrypt` 方法则是将获得的水印比特流进行解密操作，因为在嵌入水印时可能加入了水印密钥，具体代码如下：

```
def extract_decrypt(self, wm_avg):
    wm_index = np.arange(self.wm_size)
    np.random.RandomState(self.password_wm).shuffle(wm_index)
    wm_avg[wm_index] = wm_avg.copy()
    return wm_avg
```

这里首先创建了一个数组用来代表水印信息的索引。随后经过随机重排，以我们输入的水印密钥作为种子来对水印信息的索引打乱。最后将打乱后的索引应用到水印信息数组上，将其中的水印信息根据打乱后的顺序重新排列，即可实现解密操作。这个 `wm_avg` 就是最终的水印信息比特流了。最后通过 `imwrite` 方法就能够生成出对应的图像了。以上则是图像水印的提取过程。



而文字水印还需要另外进行一些额外的操作。

```
def one_dim_kmeans(inputs):
    threshold = 0
    e_tol = 10 ** (-6)
    center = [inputs.min(), inputs.max()]

    for i in range(300):
        threshold = (center[0] + center[1]) / 2
        is_class01 = inputs > threshold
        center = [inputs[~is_class01].mean(), inputs[is_class01].mean()]
        if np.abs((center[0] + center[1]) / 2 - threshold) < e_tol:
            threshold = (center[0] + center[1]) / 2
            break
```

对于 one_dim_kmeans 方法，实际上为二分聚类方法^[14]，其作用时将水印分为两类，即 0 和 1，从而提取出水印信息；同时这个方法还能确定一个适当的阈值，作为提取水印的依据，而这个阈值，即在嵌入文字水印后给出的水印长度。

```
def extract_with_kmeans(self, filename, wm_shape):
    wm_avg = self.extract(filename=filename, wm_shape=wm_shape)
    return one_dim_kmeans(wm_avg)
```

对于 extract_with_kmeans 方法则是调用了先前介绍的各种方法，其中 extract 与提取图像水印的方法公用了。经过二分聚类处理的结果，即为最后的文字水印比特流。同样经过后续的解密操作和转化操作即可得到最后的文字水印。

至此文字水印与图像水印的提取算法的设计介绍完成。

3.3 图像攻击的算法设计

数字水印技术虽然在许多应用场景中提供了额外的安全性和保护，但仍然容易受到各种攻击手法的影响。在数字水印从嵌入到提取的整个过程中，最脆弱的地方其实是图像被加密后流传于市场上后经由他人对图像的操作。如图 3-3 所示。

其中 n 为攻击手法； C_n 为经过攻击的含数字水印的图像； W_n 为受到过攻击的含数字水印图像所提取出来的水印编码。

攻击的过程经常会出现整个水印嵌入提取系统与外界接触的部分。这部分通常是整个系统最脆弱的地方，因此可能经常被攻击。所谓攻击，其实就是未授权的对图像进行操作，最常见的攻击即为图像处理攻击，其具体包含有对图像进行集合变换、旋转、缩放、翻转等这些不破坏图像的对图像进行的浅操作；另外还有如在图像中添加遮挡块、裁剪图像等这类可破坏图像的对图像进行的深操作。其次比较常见的还有对图像参数的修改攻击，如调整图像的亮度或对比度或者对图像进行色彩平衡，这样的攻击可能不会对图像造成观感上的差异，



但会导致水印变得难以察觉或提取。攻击的手法还有很多，如噪声干扰攻击、压缩攻击、替换攻击等等。这些操作都可能导致水印失真，即提取出的水印被破坏变成一串无用信息；或直接导致水印无法被提取。

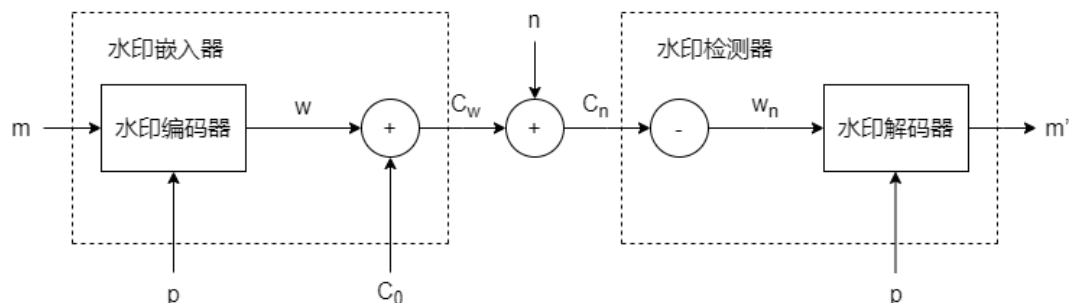


图 3-3 数字水印攻击的简单流程

日常应用中在图像经过嵌入水印后通常都会被第三方未经授权的用户进行各种操作，为模拟测试嵌入水印的安全性，系统加入了攻击模块来实时测试水印的嵌入与提取性能是否对等，即是否能够将嵌入的图像水印或是文字水印完整的提取出来。

市面上的攻击手法有非常多，将其全部实现将花费大量的时间与精力，同时集成在一个轻量级系统中并不现实，因此本系统实现的攻击手法为日常生活中用户们应用得最广泛的一部分，这部分攻击手法具有相当的普遍性，能够作为实时测评水印安全性的依据。

(1) 亮度攻击

对于亮度攻击，顾名思义即在攻击的过程中调整水印图像的亮度，对安全性能不好的水印算法，可能经过调整亮度后水印就会变得模糊甚至不能完全提取出来。而亮度攻击实际上又可以细分为亮度调高攻击与亮度调低攻击，具体设计如下所示。

```
def bright_att(input_filename, output_file_name, ratio=0.8):  
    input_img = cv2.imread(input_filename)  
    output_img = input_img * ratio  
    output_img[output_img > 255] = 255  
    cv2.imwrite(output_file_name, output_img)
```

通过将输入图像的像素值乘以给定比例的 `ratio`，从而达到调整图像亮度的目的，在这里默认是 0.8，即亮度调低攻击，当需要执行亮度调高攻击时，只需要将传入的 `ratio` 的值大于 1 即可实现。当然，本系统对于亮度调高或调低攻击的受击程度是有限的，经过测试，`ratio` 为 0.8 或 1.2 时是保证图像不失真的情况下单次调整的最大偏移值。在往下或往上的话，图像将有可能发生失真，从而导致水印被错误提取。

(2) 多遮挡攻击

接下来介绍多遮挡攻击。遮挡，即用不透明的图块来覆盖水印图像的某一个区域。这种



攻击手法在市面上极为常见。在这里本系统使用的图块即普通的白色图块，理论上不论是什么样的图块都能够用于进行攻击。具体代码如下所示。

```
def shelter_att(input_filename, output_file_name, ratio=0.1, n=3):
    input_img = cv2.imread(input_filename)
    input_img_shape = input_img.shape
    output_img = input_img.copy()
    for i in range(n):
        tmp = np.random.rand() * (1 - ratio)
        start_height, end_height = int(tmp * input_img_shape[0]), int((tmp + ratio) *
input_img_shape[0])
        tmp = np.random.rand() * (1 - ratio)
        start_width, end_width = int(tmp * input_img_shape[1]), int((tmp + ratio) *
input_img_shape[1])
        output_img[start_height:end_height, start_width:end_width, :] = 255
    cv2.imwrite(output_file_name, output_img)
```

其中 ratio 表示每个遮挡块的占比，默认为 0.1；n 表示要添加的遮挡块的数量，默认为 3 个，当然该数据是可以由用户自定义的。另外，算法使用了随机策略用于随机选择一个地方用于生成白色图块来进行遮挡，在这里 1-ratio 是为了防止溢出。

（3）旋转攻击

旋转攻击，即将图像进行旋转，由于旋转可能会导致水印信息像素偏移，从而使得水印信息不能够提取出来。具体实现如下所示。

```
def rot_att(input_filename, output_file_name, angle=45):
    input_img = cv2.imread(input_filename)
    rows, cols, _ = input_img.shape
    M = cv2.getRotationMatrix2D(center=(cols / 2, rows / 2), angle=angle, scale=1)
    output_img = cv2.warpAffine(input_img, M, (cols, rows))
    cv2.imwrite(output_file_name, output_img)
```

在这里旋转角度默认为 45 度，这个角度广义上是能够旋转的最大角度，超过这个角度通过上下翻转或是左右翻转都能使得旋转角度小于或等于 45 度。当然，这个旋转角度同样能够由用户自定义的，支持顺时针与逆时针旋转，只要分别设置旋转角度为正数或是负数即可。

（4）裁剪攻击

裁剪攻击也是市面上很常用的攻击手法之一，再将其细分的话，可以分为纵向裁剪攻击和横向裁剪攻击，其具体实现如下所示。

```
def cut_att_height(input_filename, output_file_name, ratio=0.8):
    # 纵向剪切攻击
```



```
input_img = cv2.imread(input_filename)
input_img_shape = input_img.shape
height = int(input_img_shape[0] * ratio)
cv2.imwrite(output_file_name, input_img[:height, :, :])

def cut_att_width(input_filename, output_file_name, ratio=0.8):
    # 横向裁剪攻击
    input_img = cv2.imread(input_filename)
    input_img_shape = input_img.shape
    width = int(input_img_shape[1] * ratio)
    cv2.imwrite(output_file_name, input_img[:, :width, :])
```

对于实现方法其实很容易就能看懂，前者是只保留图像的前 `height` 行像素，而后者只保留图像的前 `width` 列像素。其中，默认的裁剪距离 `ratio` 为 0.8。

(5) 缩放攻击

接着来介绍缩放攻击，同样是市面上常用的攻击手法，通过调整画布的宽与高来达到攻击的目的，其具体实现如下所示。

```
def resize_att(input_filename, output_file_name, out_shape=(500, 500)):
    input_img = cv2.imread(input_filename)
    output_img = cv2.resize(input_img, dsize=out_shape)
    cv2.imwrite(output_file_name, output_img)
```

默认的缩放尺寸是宽高各为 500 像素。当然这个缩放宽高也是可以由用户自定义的。

(6) 椒盐攻击

椒盐攻击是数字图像处理中常见的一种图像噪声，也被称为脉冲噪声，或是可以简单理解为就是一种噪声攻击。这种噪声模式的特点是再图像中随机分布出现黑色和白色的噪点，即所谓的椒与盐，这样操作可以为图像带来不可忽视的干扰^[15]。椒盐攻击通过人为去添加的情况其实比较少，大部分情况都是再图像采集设备、传输信道或图像存储过程中的干扰引起的。其具体实现算法如下所示。

```
def salt_pepper_att(input_filename, output_file_name, ratio=0.01):
    input_img = cv2.imread(input_filename)
    input_img_shape = input_img.shape
    output_img = input_img.copy()
```



```
for i in range(input_img_shape[0]):  
    for j in range(input_img_shape[1]):  
        if np.random.rand() < ratio:  
            output_img[i, j, :] = 255
```

其中 `ratio` 的值代表的是椒盐概率，或称为椒盐比例。这里同样采用了随机策略，在图像中随机的添加黑色像素点与白色像素点。对于椒盐概率也是能够由用户模拟控制的。

3.4 人脸分割算法设计

深度学习模型 Deeplab 系列是分割领域的经典算法，它结合了经典的空间金字塔模块和编码-解码结构，前者通过在不同速率和多个有效视场范围内对输入特征进行过滤或者池化操作，能够对多尺度的上下文特征进行编码，后者则通过逐步恢复空间信息来捕获更锐利的边界信息。

DeeplabV3Plu 是一种用于语义分割任务的深度学习模型，它是 DeepLab 系列模型的一种改进版本。它引入了引入空洞卷积，利用空洞卷积来扩大感受野，以更好地捕捉图像中的上下文信息。传统的卷积操作只关注局部区域，而空洞卷积通过在卷积核中引入间隔（或称为膨胀率），使得卷积核能够跳过一些像素点，从而扩大感受野。其次，它使用了 ASPP 模块来处理不同尺度的信息。ASPP 模块使用多个并行的空洞卷积分支，每个分支具有不同的膨胀率，以捕捉来自不同感受野的特征。最后，将这些特征进行汇总并进行融合，以生成更丰富的特征表示。为了结合低层次的细节特征，DeepLabV3Plus 引入了一个编码器-解码器结构。在编码器部分，通过堆叠多个残差块和降采样操作，提取高层次的语义特征。然而，这会导致空间信息的丢失。因此，在解码器部分，使用反卷积（或上采样）操作来恢复特征图的分辨率，并与对应的低级特征进行融合。为了进一步提升融合的效果，DeepLabV3Plus 引入了注意力机制。该机制利用辅助监督信号和空间注意力模块，自适应地对不同的特征图进行加权融合。这样可以使网络更加关注重要的特征区域，提升语义分割的准确性。通过改进上述机制，它能够准确地标记图像中每个像素的类别，从而在许多计算机视觉领域（如图像分割、自动驾驶等）中发挥着重要作用。整体网络结构如图 3-4 所示。

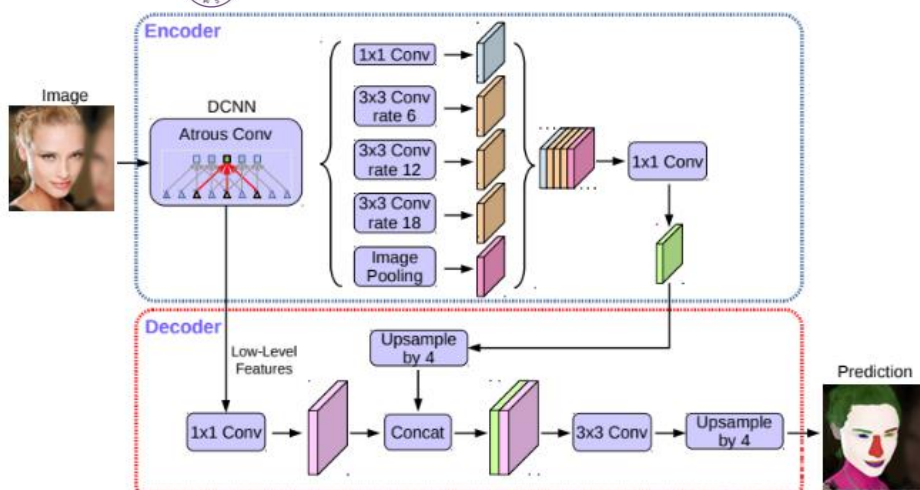


图 3-4 人脸分割模型

4 界面设计

4.1 GUI 界面的设计

本系统采用了 GUI 图形界面，主要目的是为了更方便用户的使用。通过简单的点击按钮事件就能够达到嵌入与提取数字水印的目的。系统 GUI 界面主要分为以下模块：图像上传及显示模块、图像水印以及文字水印的嵌入模块、图像水印以及文字水印的提取模块、对图像进行攻击模块。

首先是图像上传显示模块，我们需要将上传的文件显示到 GUI 中以实时的观测图像对应的变化。对此这里创建了一个 `ImageArea` 类用于显示图像，并且在其中专门定义了一个 `update_image` 方法，以实时的更新图像。伪代码实现如下所示。

```
class ImageArea:
    def __init__(self, root, text, x, y1, y2, size):
        self.root, self.text = root, text
        self.x, self.y1, self.y2, self.size = x, y1, y2, size
        # 定义显示在图像上方的说明文字框 text_label
        # 定义画布 canvas 并让画布呈现为灰色
        self.gray_background = ImageTk.PhotoImage(Image.new("RGBA", (size, size),
"gray"))
        self.image_on_canvas = self.canvas.create_image(0,
            0, anchor=tk.NW, image=self.gray_background)

    def update_image(self, file_path, target_canvas):
        if file_path:
            original_image = Image.open(file_path).convert("RGBA")

            width, height = original_image.size
```



```
target_size = (self.size, self.size)
# 计算宽度和高度的缩放因子, 并根据最小缩放因子以使图像适合目标尺寸

width_scale = target_size[0] / width
height_scale = target_size[1] / height
# 选择最小缩放因子以使图像适合目标尺寸
scale_factor = min(width_scale, height_scale)
# 根据比例因子 scale_factor 计算新尺寸, 得到 new_width 与 new_height
# 调整图像的大小并更新 target_canvas 为 image_tk
```

有关能够上传的图像, 本系统暂时只提供上传最常见的三种图像形式: png、jpg、jpeg。

具体代码实现如下所示。

```
def upload_img(self, target_area):
    file_path = filedialog.askopenfilename(filetypes=[("Image files", "*.png;*.jpg;*.jpeg")])
    # 获取文件路径
    if file_path:
        print(file_path)
        if target_area == self.image_area1:
            # image_area1 为原图的 ImageArea 对象
            self.ori_img_path = file_path
        elif target_area == self.image_area3: # image_area3 为图像水印的 ImageArea 对象
            self.wm_img_path = file_path
        target_area.update_image(file_path, target_area)
```

这里只呈现出图像水印的上传方法, 而考虑到可能会有用户需要嵌入文字水印, 在这里只需要定义一个输入框用来获取用户输入即可, 由于代码较简单这里未展示。

然后为图像水印以及文字水印的嵌入模块。这里定义了一个 `ori_img_path` 作为全局变量存储原始图像的路径; 同样, 定义了 `wm_img_path` 存储水印图像的信息; 另外还定义了 `wm_text` 存储文字水印的信息。当用户点击了嵌入水印按钮后, 会弹出一个 `password_window` 用来提醒输入或选择不输入密码。当点击 `password_window` 窗口中的确定按钮后, 会经过一系列的逻辑判断来嵌入数字水印, 由于代码较长以下将用伪代码显示, 算法的伪代码如下所示。

```
def embed_img_wm(self, password, password_window):
    try:
        initial_dir = os.path.expanduser("~/Desktop")
        # 首先判断是文字水印还是图像水印
        wm_text = self.text_area1.entry.get()
        if 原始图像路径为空:

            提醒: 请先上传需要嵌入的图像
        else:
```



```
if 水印图像路径为空:
    if 文字水印为空:
        提醒: 请先上传需要嵌入的文字水印或图像水印
    else:
        if 密码为空:
            bwm = WaterMark(不传入密码)
            bwm.read_img(self.ori_img_path)
            bwm.read_wm(wm_text, mode='str')
            bwm.embed(f'{initial_dir}/wm_img.png')
            # 更新图像
            self.image_area2.update_image(f'{initial_dir}/wm_img.png',
                                           self.image_area2)

            len_wm = len(bwm.wm_bit)
            提醒: 水印嵌入成功, 生成水印长度
        else:
            pwd = eval(password)
            bwm = WaterMark(传入密码)
            下面步骤与密码为空时类似
else:
    if 文字水印不为空:
        提醒: 文字水印和图像水印只能有一个, 请重试
    else:
        if 密码为空:
            bwm = WaterMark(不传入密码)
            bwm.read_img(self.ori_img_path)
            bwm.read_wm(self.wm_img_path)
            bwm.embed(f'{initial_dir}/wm_img.png')
            # 更新图像
        else:
            pwd = eval(password)
            bwm = WaterMark(传入密码)
            下面步骤与密码为空时类似

except Exception as e:
    提醒: 错误
```

接下来为图像水印以及文字水印的提取模块。实际提取水印的过程与嵌入过程是类似的, 点击按钮后会出现一个选择窗口, 让用户选择是提取文字水印还是图像水印, 若为图像水印则只需要选择输入或不输入密码点击确定后即可进入逻辑判断来调用提取水印程序, 而若为文字水印则还需要额外输入水印长度这一变量。其他步骤与嵌入水印类似, 将图像更新区域换为提取水印区域 `image_area4`, 而如果是文字水印的话则会更新输入框为水印信息。以下将不再详细介绍。



最后一个为对图像进行攻击模块。攻击手法都与 GUI 上的各个按钮相绑定，本系统实现的攻击手段有亮度调低攻击、亮度调高攻击、多遮挡攻击、旋转攻击、椒盐攻击、横向裁剪攻击、纵向裁剪攻击、缩放攻击。下面以旋转攻击作为例子，其他攻击手法的实现与其类似，根据各个攻击的不同会有不同的调整。伪代码如下所示。

```
def rotation_att(self):
    initial_dir = os.path.expanduser("~/Desktop")
    try:
        初始化窗口
        提醒框
        输入旋转角度

        def button_click():
            try:
                att.rot_att(f"{initial_dir}/wm_img.png", f"{initial_dir}/wm_img.png",
                           angle=eval(input_var.get()))
                self.image_area2.update_image(f"{initial_dir}/wm_img.png",
                                               self.image_area2)
                提示：攻击成功！
            except Exception as e:
                提示错误
        定义初始化开始按钮并调用 button_click 方法
        定义取消按钮
        设置窗口居中
    except Exception as e:
        提示错误
```



4.2 系统的测试与评估

首先需要测试系统的功能，为了确保系统的每个功能都能正常使用，本章将对系统三大板块的功能进行测试，以更好的对系统错误功能进行修正；同时为了评估系统的具体能力，本章将从数字水印嵌入与提取过程中常用的三个指标，即可读性、鲁棒性和安全性三个方面来对该系统进行评估，从而尽可能地从中找到系统设计的不足之处从而加以改进。

4.3 系统上传功能测试

为了让用户能够通过简单的操作来实现对应功能，本系统采用了如下的 GUI 主窗口设计。只需要通过按下相应的按钮即可，在每个按钮内部还有相应的提示来提醒用户操作。GUI 主窗口如图 4-1 所示。



图 4-1 系统 GUI 主窗口展示

对于上传功能，用户只需要点击相应的“上传图像”或“上传水印”，即可打开资源管理器，让用户自主选择自己想要上传的图像。而为了能够直观的体现用户选择的图像，在



GUI 主界面中的原图像窗口和图像水印窗口会分别显示用户选择的两张图像,如图 4-2 所示。

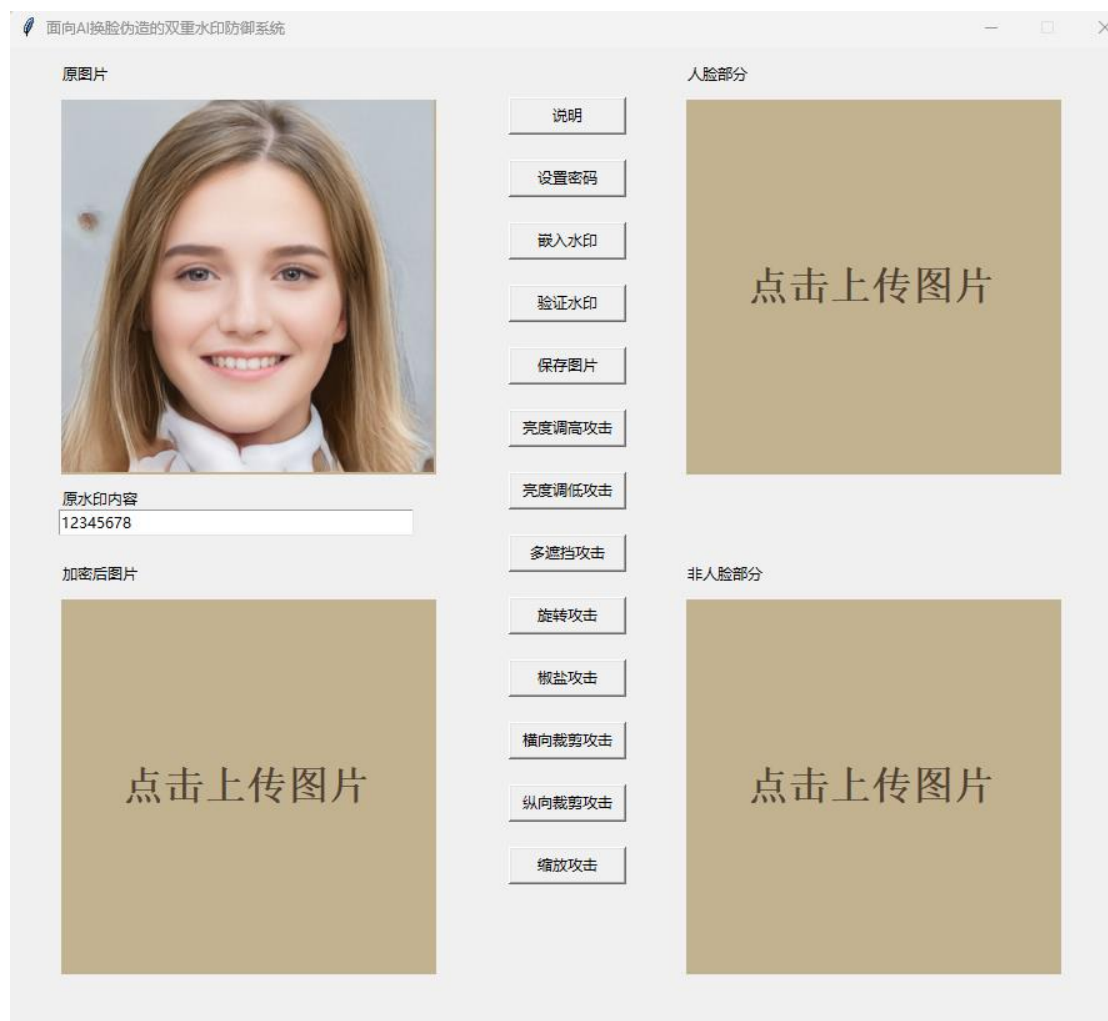


图 4-2 原图像与水印图像的上传

4.4 系统嵌入与提取水印功能测试

对于嵌入水印,用户在上传了原图像与水印图像或者使用文字水印后,就可以通过直接点击“嵌入水印”按钮来进行水印的嵌入,这时会弹出一个提醒框,用来提醒用户输入密码,并且说明如果不设置密码可以直接点击确定,如图 4-3 所示。

当用户输入密码或选择不适用密码,点击确定按钮后,就会开始水印的嵌入过程。

另外需要说明的是,嵌入水印提供了最长 8 位自定义的数字输入,如果超过 8 位数字,会进行弹窗提示。8 位数字可以是操作者自定义,该水印信息在同时嵌入到图像的人脸部分和非人脸部分。因此,在提取水印的时候,会先对水印图像进行人脸分割,然后同时提取人脸部分和非人脸部分的水印,并将提取到的水印信息显示在图像的下方,如图 4-4 所示。

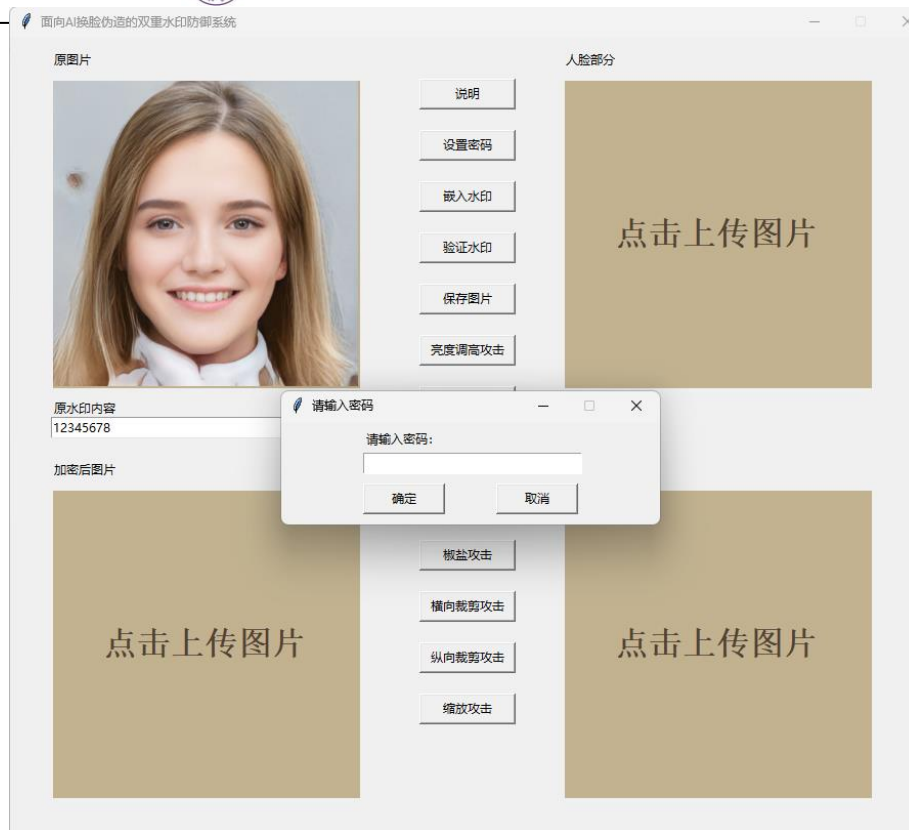


图 4-3 输入密码提醒框

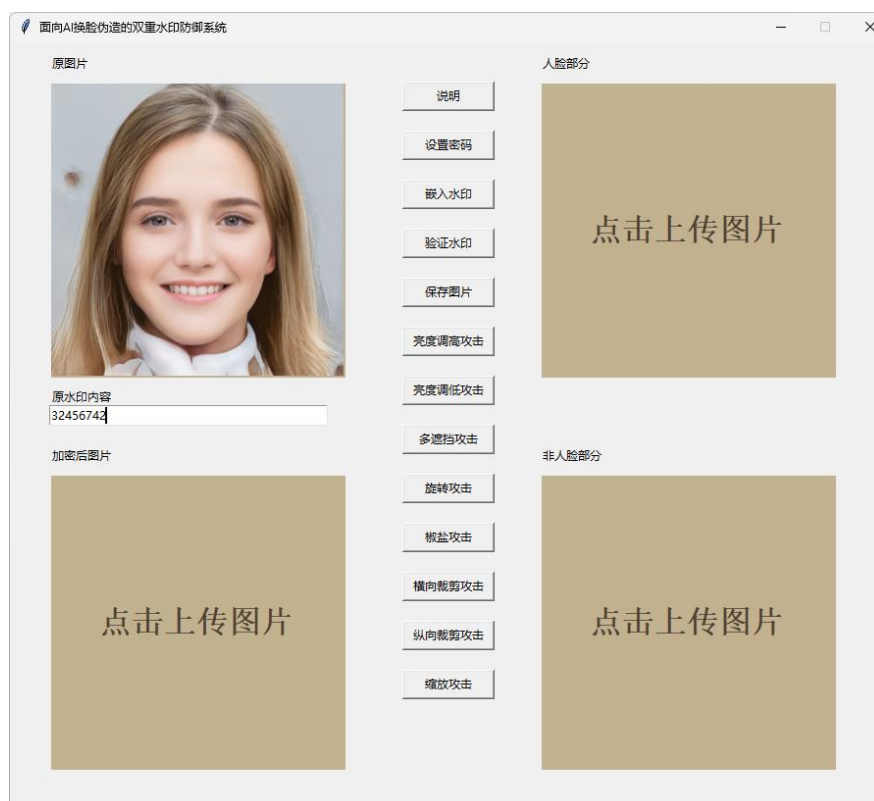


图 4-4 数字水印输入

点击“嵌入水印”后，水印会同时嵌入到人脸部分和非人脸部分，并在图像下方显示提



取到的水印信息，同时弹窗“加密成功”，如如图 4-5 所示。

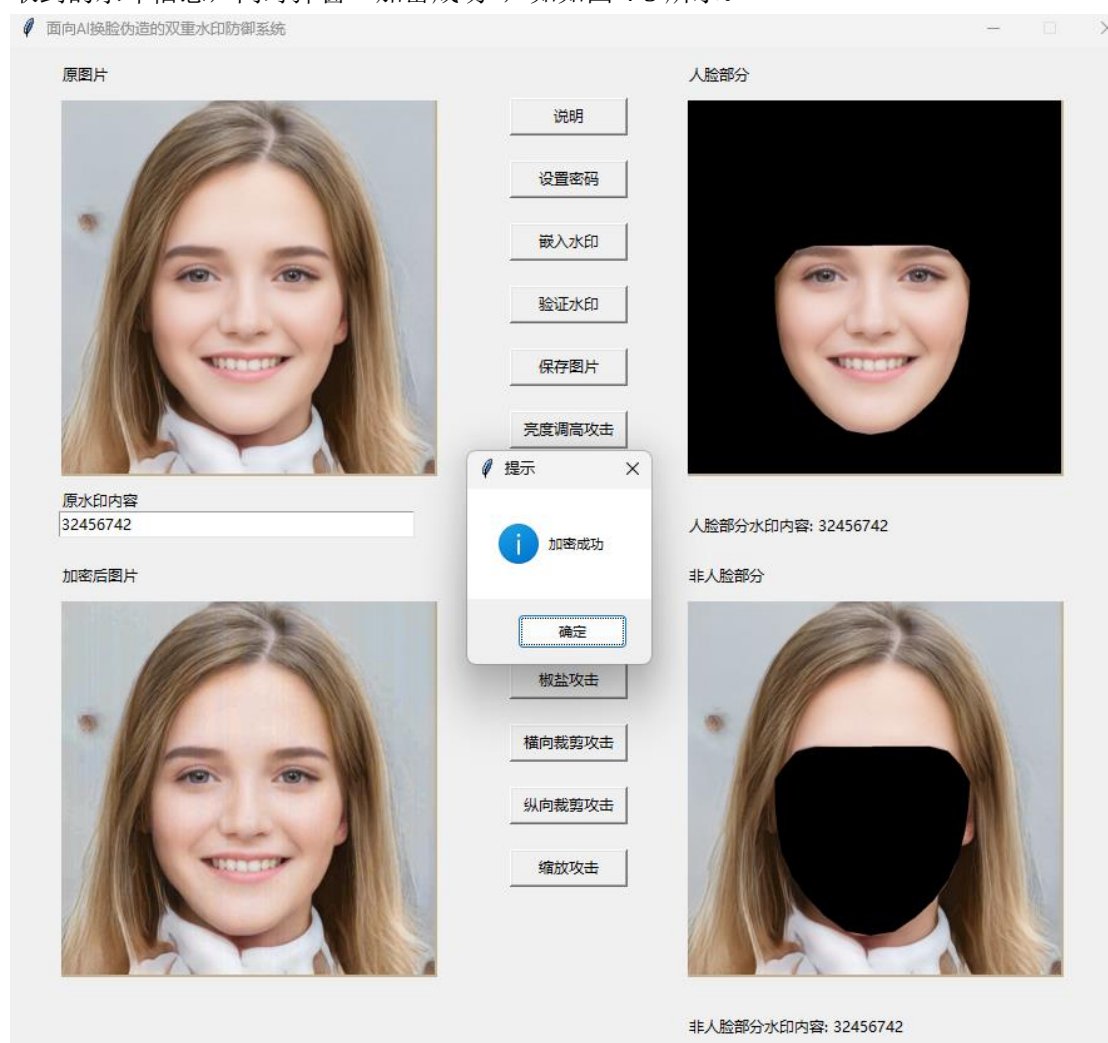


图 4-55 数字水印嵌入成功

至此，水印的嵌入过程与提取过程介绍完毕。

4.5 系统攻击功能测试

对于诸多的攻击功能，下面将以多遮挡攻击为例来进行介绍，其他的攻击功能流程类似。

当点击“多遮挡攻击”按钮时，会出现一个提醒框用来提醒输入遮挡块的数量，如图 4-8 所示。

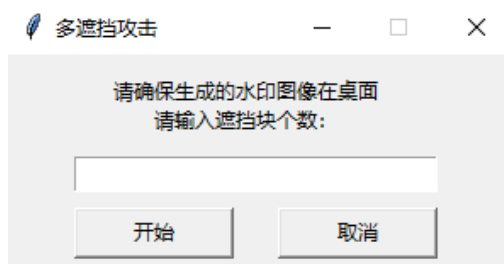


图 4-8 多遮挡攻击提醒框



这里由用户随意输入，这里以 20 为例，即代表在税后生成的图像中会含有 20 个随机位置的遮挡块。点击“开始”即可进行攻击。如图 4-9 所示。

对于其他攻击功能，其实都与所演示的多遮挡攻击类似，只是出现的提醒框不同，让用户输入的参数有不同。

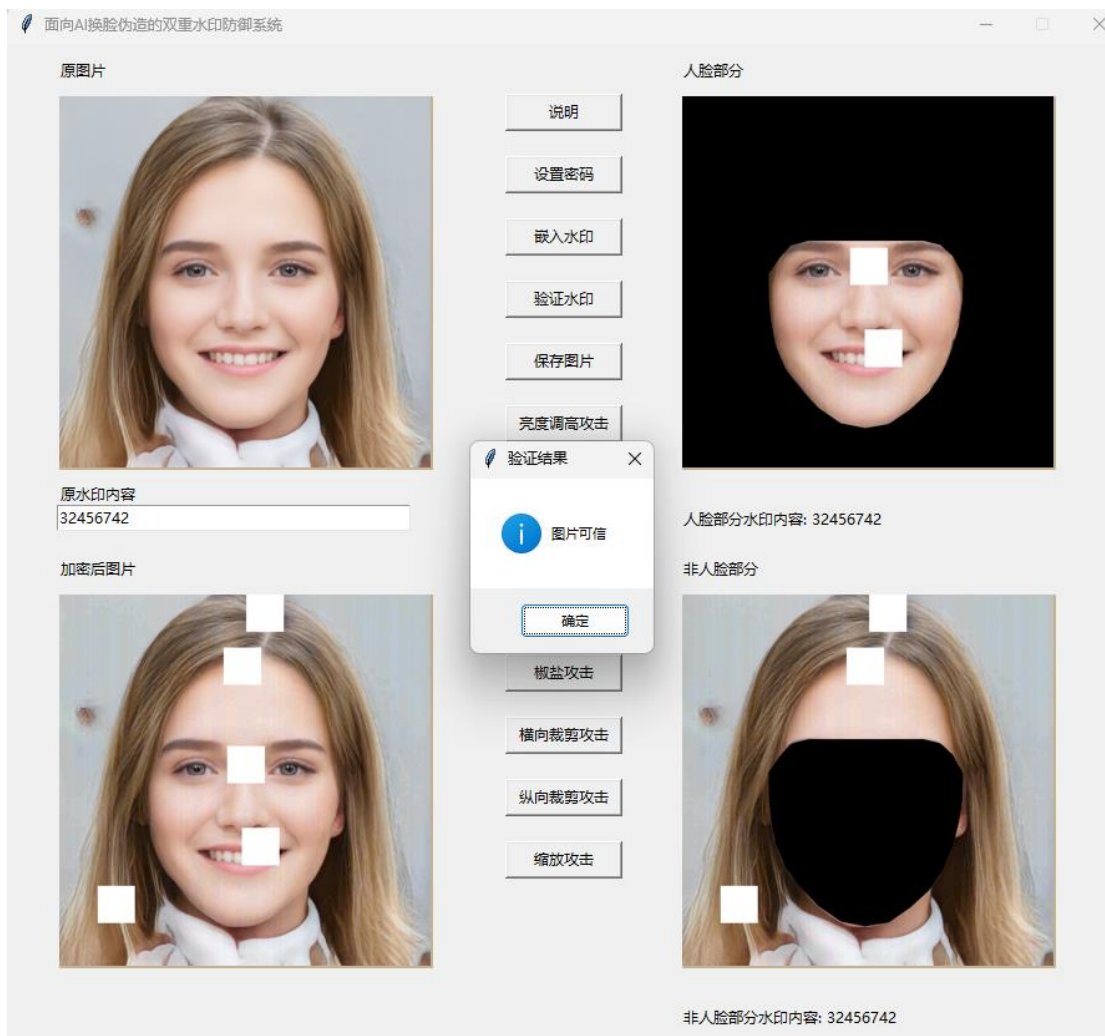


图 4-9 多遮挡攻击成功

4.6 测试结果与评估

经过测试，对测试结果进行分析后，可以发现本系统能够适应大部分的应用场景。如图 4-10所示。



图 4-10 原图像(左)与嵌入数字水印图像(右)

其中左图是原始图像，而右图是嵌入过水印的图像，经由人的肉眼实际上已经比较难发现，可读性较差，因此本系统的隐匿性已经达到失眠上的水准。

对于水印图像我们经过系统嵌入的各种攻击手法进行攻击后都是能够提取出相应水印的，因此本系统的鲁棒性与安全性能达到了市面上的水准。

综上所述，本系统已经能够较为良好的进行水印的嵌入与提取，并达到了预期的要求。

4.7 小结

本章以系统模块进行分类根据，对系统中的各个模块功能进行了测试，同时展示了各个功能的模块的实际运行图，最后对测试结果进行了评估，从测试结果来看可以得到结论：本系统的各个功能模块基本达到了预期的要求。



5 用户操作简介

5.1 设置密码功能

软件的密码设置功能主要是为了保护嵌入水印后的图像。如果软件使用者想对某张水印图像进行水印提取，则需要输入密码，否则，不允许提取水印。因此，在每次对图像嵌入水印之前，都必须设置密码，否则会弹窗提示。设置密码的界面如图 5-1 所示。



图 5-1 密码设置界面

5.2 图像加载功能

将原始图像加载到软件。首先点击左上角方框任意地方，会弹窗需要加载图像的路径，选中待加载的图像，双击即可将原始图像加载。图像加载界面如图 5-2 所示。



图 5-2 原始图像加载界面

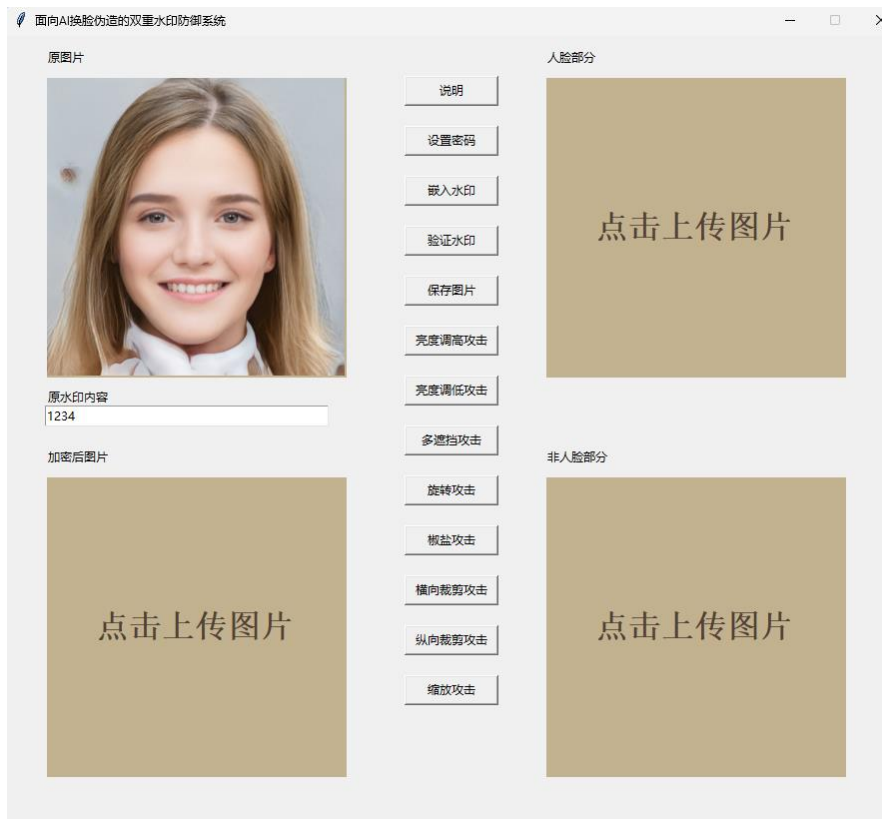


图 5-3 原始图像加载成功界面



5.3 水印自定义功能

水印设置最长 8 位数字，可以有使用者自定义输入，用于嵌入到原始图像中。输入框在原图片的下方，单击输入框即可输入。自定义输入水印如图 5-4 所示。

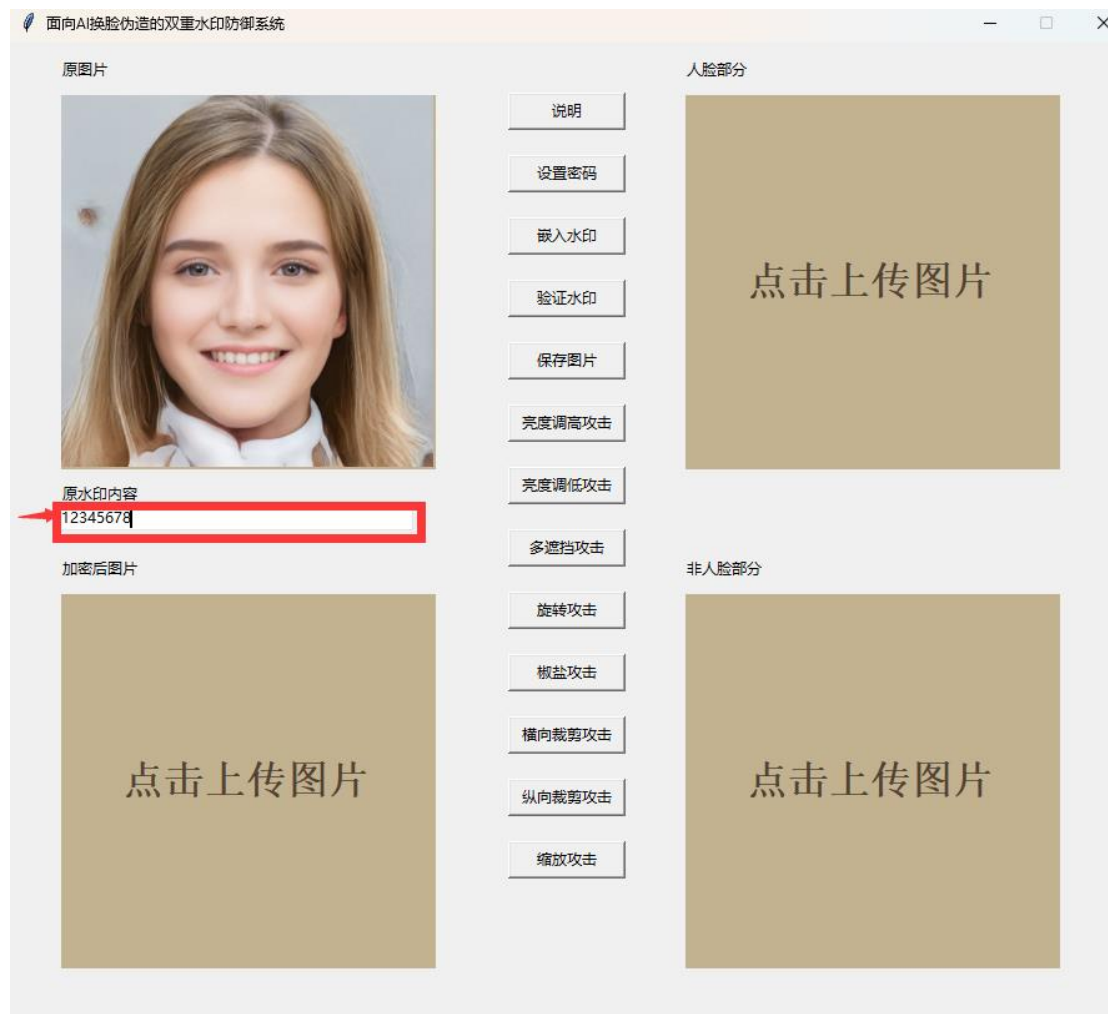


图 5-4 自定义水印内容输入界面

5.4 嵌入水印功能

设置好自定义的水印内容后，单击嵌入水印，此时软件会先对原始图像进行人脸分割，并将分割后的人脸部分和非人脸部分进行显示，如图 5-5 所示。其次，会同时在人脸部分和非人脸部分嵌入上述设置好的水印内容，并将嵌入水印的图像在左下方框内进行显示，用于使用者比对嵌入水印图像与原始图像在主管视觉上做对比。最后，会弹窗提示“加密成功”，并将人脸部分的水印信息和非人脸部分的水印信息在图像下方进行显示，如图 5-6 所示。

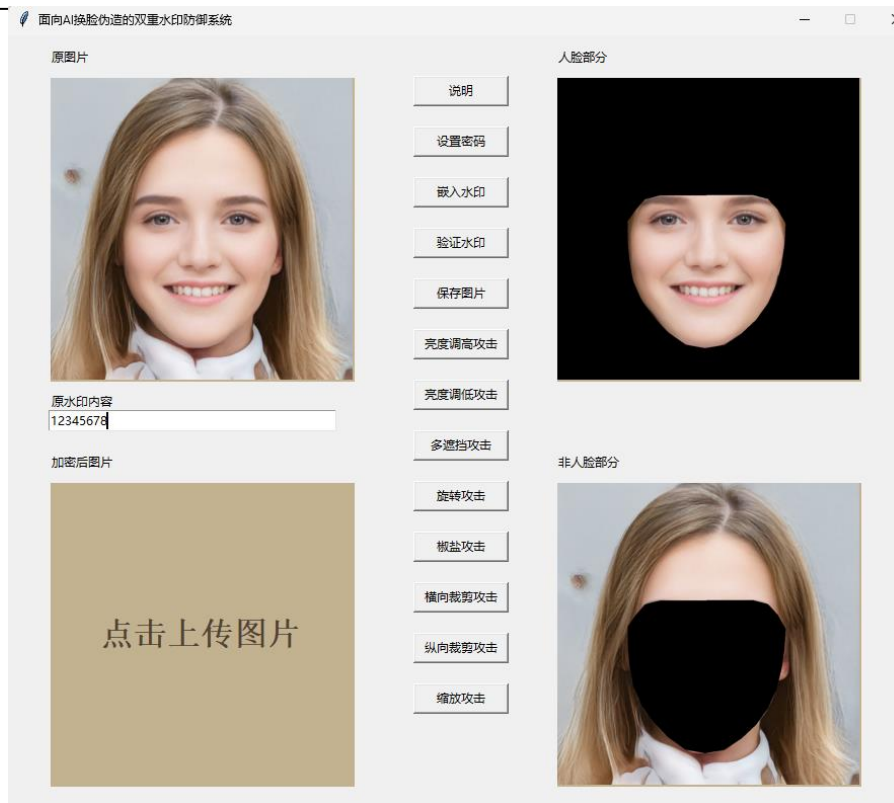


图 5-5 原始图像人脸分割界面

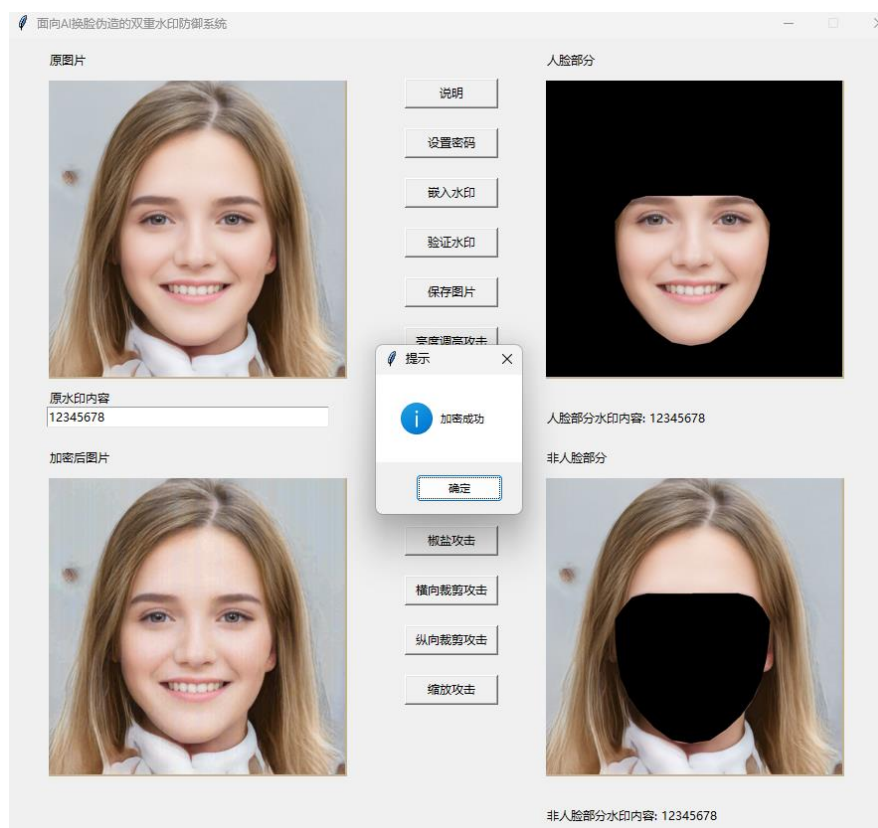


图 5-6 水印嵌入成功界面



5.5 验证水印功能

水印嵌入原始图像后，图像可能会进行传输或者遭遇攻击，此时需要对水印图像的鲁棒性进行测试，即在进行传输或者遭遇攻击后，水印图像中的水印内容是否还能否正常的提取。加载水印图像，并点击“验证水印”，首先会对水印图像进行人脸分割，然后分别提取人脸部分和非人脸部分的水印内容，并显示在图像的下方。如水印验证如图 5-7 所示。

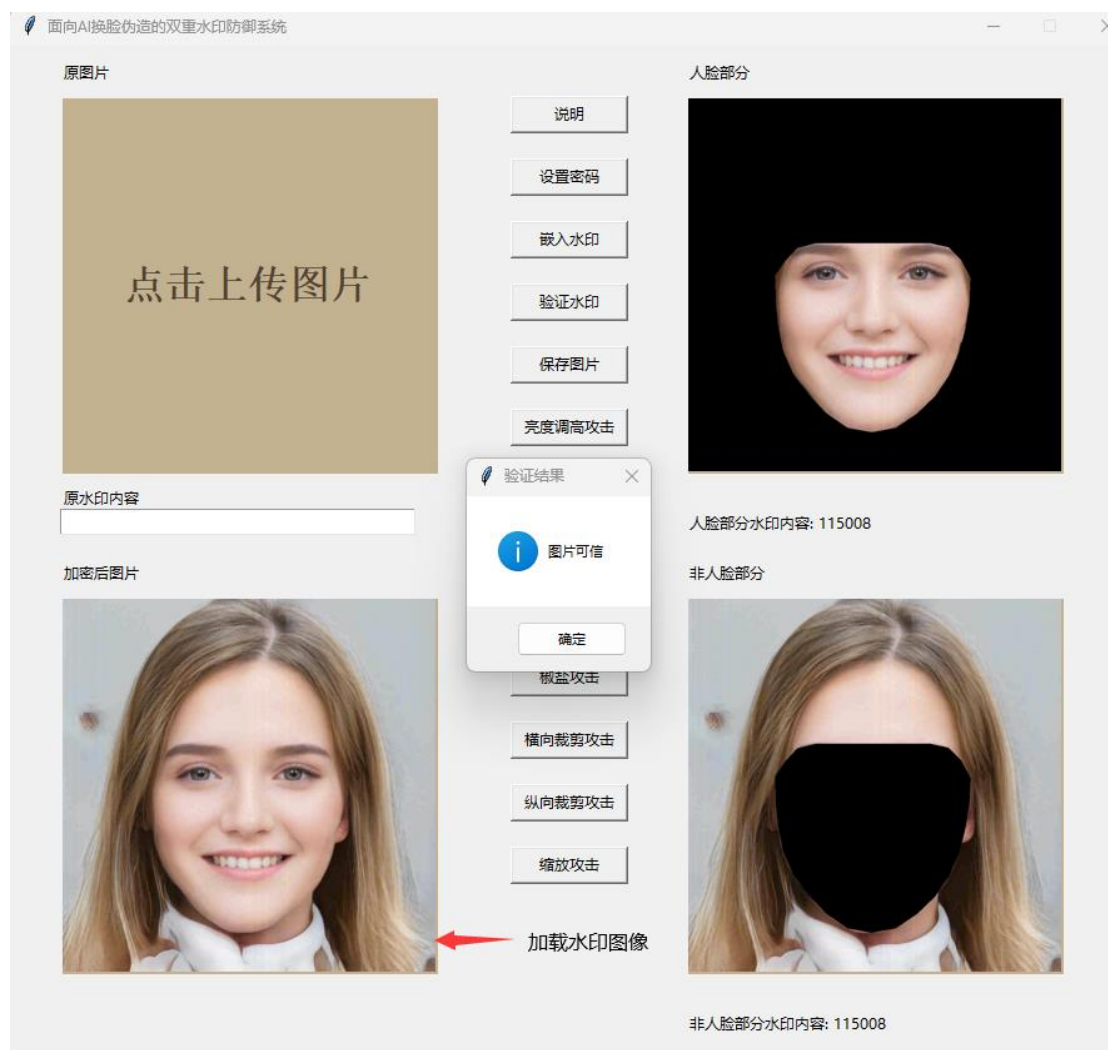


图 5-7 水印验证成功界面

5.6 保存图片

保存图片主要是用于将水印嵌入到原始图像后对该图像进行保存，默认保存在本地桌面，文件名默认是“年月日_时分秒.png”。



5.7 图像攻击测试功能

图像攻击测试主要用于模拟图像在传输过程中可能遭遇的图像操作，目前集成了亮度调高攻击、亮度调低攻击、多遮挡攻击、旋转攻击、椒盐攻击、横向裁剪攻击、纵向裁剪攻击以及缩放攻击，这些攻击主要针对水印图像，执行某种攻击之后，其攻击效果会进行显示，然后对攻击后的图像进行人脸分割，并对人脸部分和非人脸部分进行水印提取，并将水印内容显示在图像下面，同时根据提取到的水印内容进行二级水印验证，最终得出水印是否可信的结论。

以遮挡攻击为例，点击“遮挡攻击”，会弹窗“请输入遮挡块个数”，输入4后，遮挡后的图像会显示在原始水印图像位置，如图5-8所示。

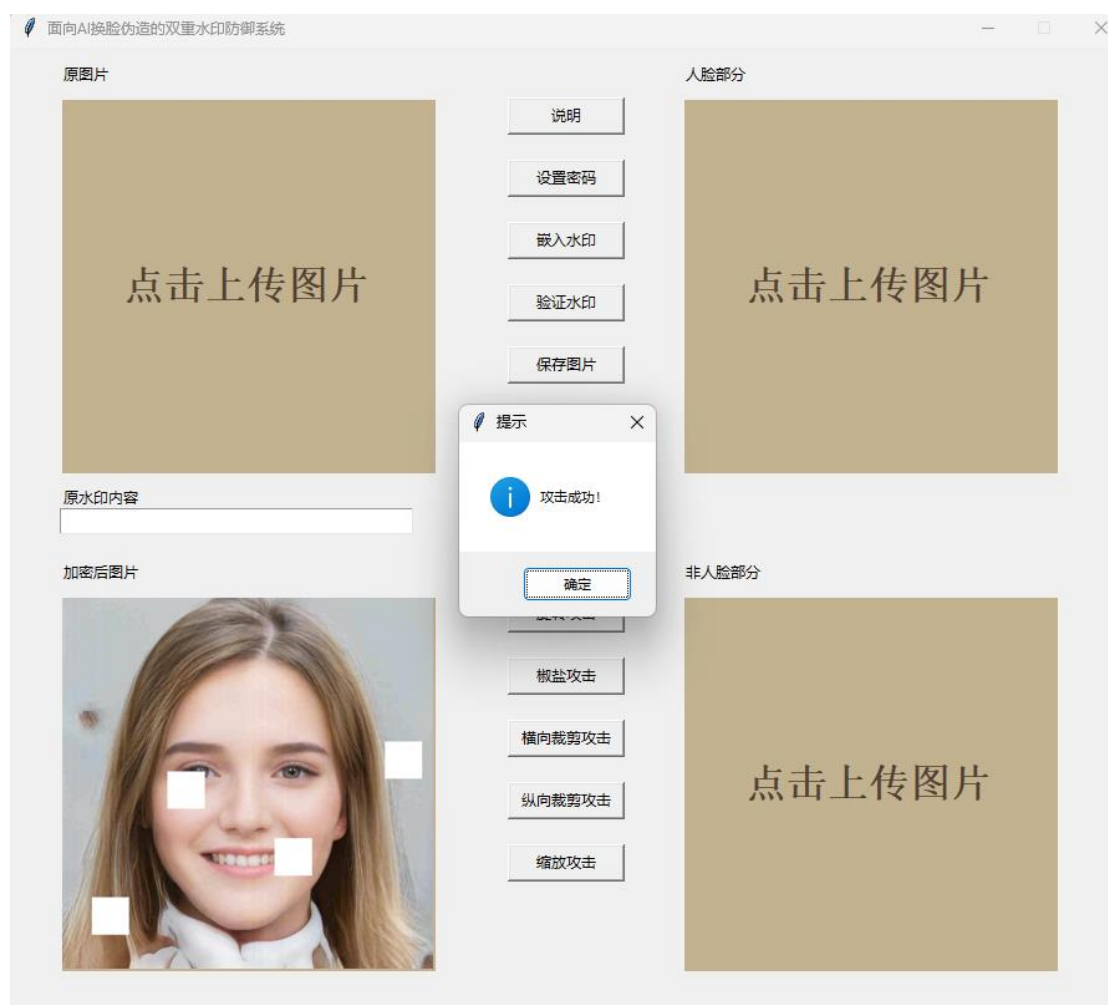


图 5-8 多遮挡攻击界面



其次，点击“验证水印”，被攻击后的水印图像会进行人脸分割，并将分割后的人脸部分和非人脸部分分别显示，同时对上述两部分的水印信息进行提取，将提取到的水印内容进行显示，并通过水印信息进行二级验证，得到图像是否可信的结论，并弹窗显示。如图 5-9 所示。

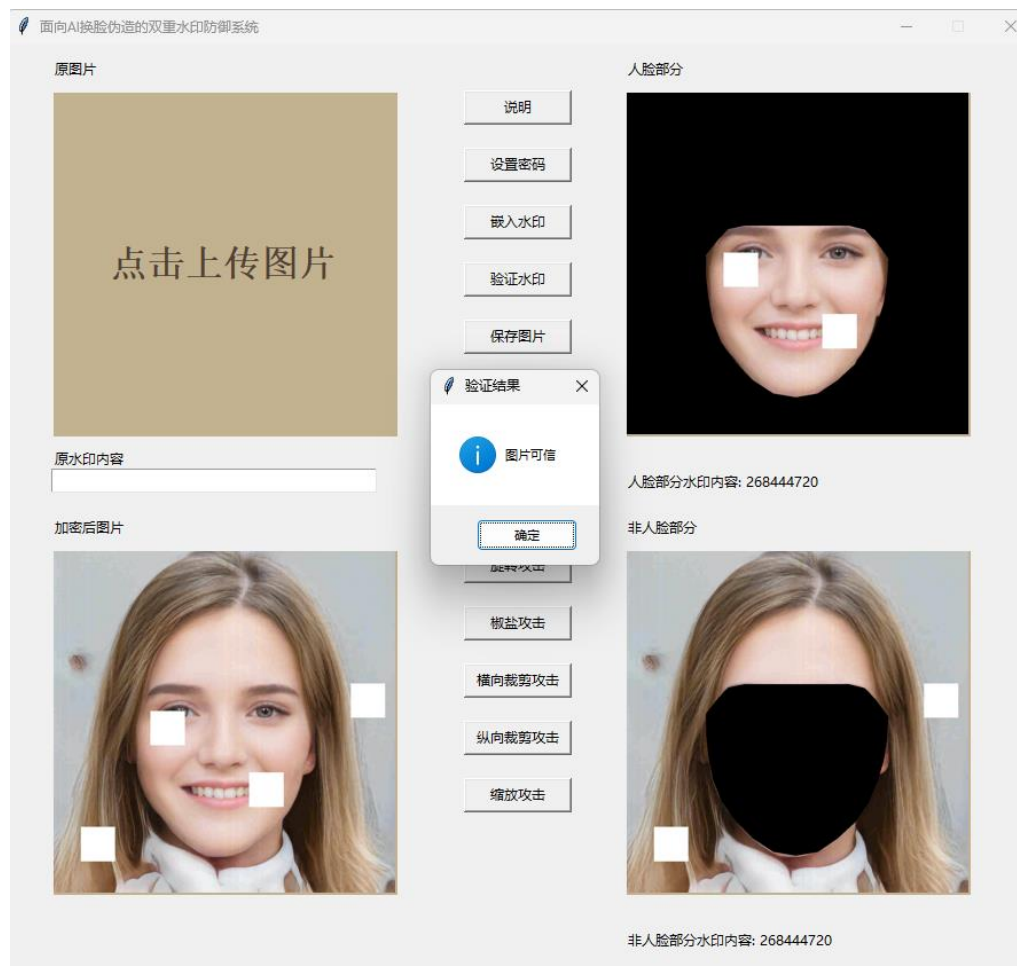


图 5-9 多遮挡攻击后水印验证界面

其他的攻击功能操作类似。整体的软件操作介绍完毕。

5.8 AI 换脸工具

目前网上可以找到各式各样的 AI 换脸工具，主流效果较好的应用有 Deepswap、Face Swap By BasedLabs、AI Face Swap By Vidnoz、Photo By Lighttricks、HeyEditor、Faceswapper、Icons8 FaceSwapper、Pixble、Swapface、deepar、ZAO 等。

需要说明的是，我们的系统主要针对换脸过程中整张人脸的替换。由于不同的应用换脸的算法不一样，如有些换脸算法主要是将鼻子、眼睛等关键部位进行替换，导致在换脸过程中水印信息可能被大部分擦除，针对此类换脸算法，我们的应用目前不能这种换脸之后的图像进行正确的水印提取。具体的换脸与验证水印过程如下：



1. 对原始图像进行水印嵌入，如图 5-10 所示。

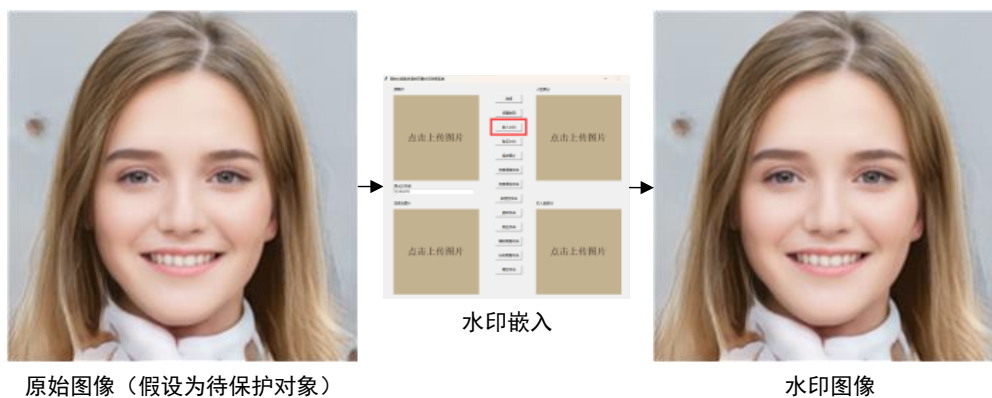


图 5-10 水印嵌入

2. AI 换脸 <https://remaker.ai/face-swap-free/>，如图 5-11 所示。

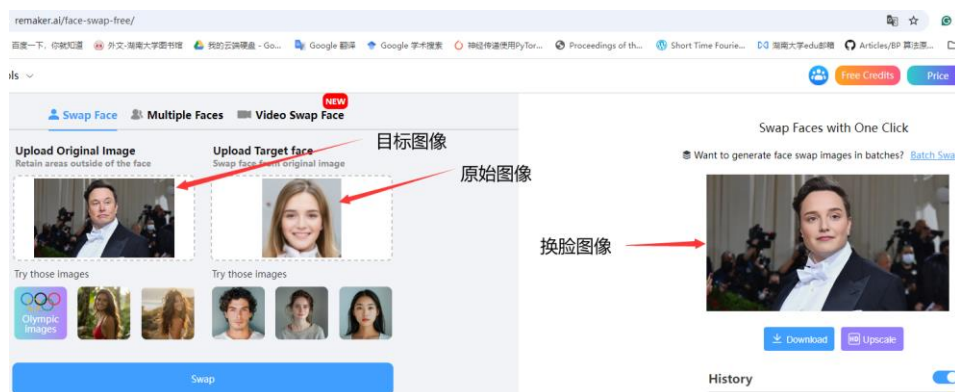


图 5-11 AI 换脸

3. 验证水印

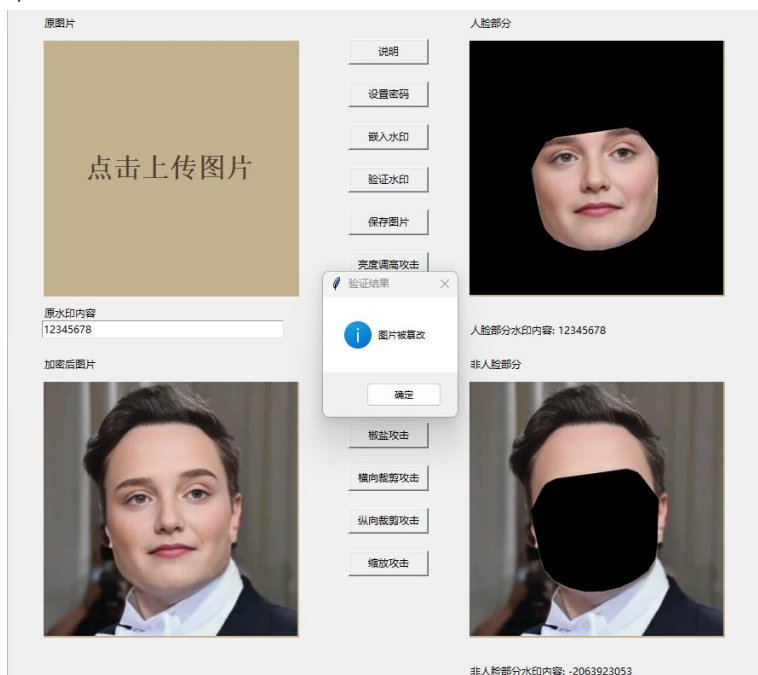


图 5-12 换脸图像水印验证



验证水印显示:脸部区域水印提取成功,且和原始水印一致。非脸部区域水印提取失败。
弹窗“图像被篡改”。